

IoT Wearable Fitness Monitoring System

Real-Time Biometric Acquisition with Cloud-Relayed Data Pipeline and Machine Learning Workout Effectiveness Prediction

Project Documentation

Authors: Toma Ionut-Adrian, Boros Fabian

University Polytechnic of Timisoara

2026

Github(source code): [fitness-monitoring-system](#)

Website: [FitMonitor — AI Workout Analysis](#)

Table of Contents

1. Introduction
 - 1.1 Project Overview
 - 1.2 Motivation and Requirements
 - 1.3 Personal Contributions
2. Design
 - 2.1 System Architecture
 - 2.2 Hardware Architecture
 - 2.3 Software Architecture
 - 2.4 AI / ML Architecture
3. Implementation
 - 3.1 Hardware Setup and Sensor Integration
 - 3.2 Edge Device Software (Raspberry Pi)
 - 3.3 Cloud Infrastructure (AWS)
 - 3.4 Backend (FastAPI on Render)
 - 3.5 Frontend (React on Vercel)
 - 3.6 Machine Learning Pipeline
 - 3.7 AI Coach (Gemini Integration)
 - 3.8 Metrics: Power, Range, and Protocols
 - 3.9 Difficulties Encountered
4. Testing

- 4.1 Sensor Testing and Calibration
 - 4.2 End-to-End Pipeline Testing
 - 4.3 ML Model Evaluation
 - 4.4 Dashboard and API Testing
 - 5. How I Used AI
 - 5.1 LLM Assistance (AI Coding Tools)
 - 5.2 Machine Learning (XGBoost Classifier)
 - 6. Conclusion and Future Work
 - 6.1 Summary
 - 6.2 Limitations
 - 6.3 Future Work
-

Abstract / Executive Summary

Presentation

This project implements a full-stack IoT wearable fitness monitoring system that captures real-time biometric data from a custom-built sensor belt worn during gym workouts. The system continuously measures three classes of physiological signals — electrocardiogram (ECG) for heart rate, electromyography (EMG) for muscle activation, and tri-axial accelerometry (MPU6050) for repetition counting — using sensors connected to a Raspberry Pi 4 Model B. At the conclusion of each workout session, the edge device computes nine aggregate features from the raw sensor streams and transmits them to the cloud via AWS API Gateway. An AWS Lambda function relays the data to a FastAPI backend deployed on Render, which persists it in a Supabase-managed PostgreSQL database. A React-based web dashboard (deployed on Vercel) allows the user to review workout data, select the workout type, and trigger an AI-powered effectiveness analysis. The AI component is an XGBoost multi-class classifier trained on approximately 51,000 synthetic workout samples, which predicts workout effectiveness on a four-tier scale (Low, Moderate, High, Maximum) with per-prediction SHAP explanations. An integrated AI Coach powered by Google Gemini 2.5 Flash provides personalized, context-aware training advice based on the analyzed workout and user profile.

Limitations

The primary limitation of the current system is its nature as a hardware Proof of Concept rather than a finished commercial wearable. We intentionally chose to build the system from raw sensors rather than using a pre-made belt to demonstrate low-level hardware integration; however, this results in conflicting physical requirements. The ECG and EMG sensors require near-zero movement noise for clean signals, while the MPU6050 explicitly tracks dynamic movement. Although the hardware is enclosed in a custom box with tightly managed cabling, the user is practically restricted to stationary exercises like push-ups and cannot move freely around a gym. Additionally, the Raspberry Pi requires a continuous

AC power connection, the EMG sensor adds bulk with its dual 9V battery requirement, and the shared I2C bus between two devices (ADS1115 and MPU6050) can cause contention. Finally, the Render free tier introduces cold-start latency during the cloud relay.

Future Works Overview

Future development could introduce AWS IoT Core with MQTT for real-time streaming of live biometric data to the dashboard, local data buffering on the Pi for offline operation, multi-device support with unique device registries, and an automated model retraining pipeline triggered by data drift detection. Migrating from the free-tier hosting infrastructure to a more robust cloud deployment (e.g., AWS ECS/Fargate) would improve reliability and eliminate cold-start issues.

1. Introduction

1.1 Motivation

Wearable fitness technology has become ubiquitous, yet commercial devices (smartwatches, fitness bands) provide opaque, proprietary metrics with no transparency into how workout quality is assessed. This project was motivated by the desire to build a transparent, end-to-end IoT system that: (a) captures raw biological signals directly from the human body using medical-grade sensors; (b) processes those signals with documented, reproducible algorithms; (c) transmits data through a proper cloud IoT pipeline; and (d) provides AI-powered workout analysis with full explainability — showing the user exactly *why* their workout was rated at a particular effectiveness level.

The project specifically targets resistance training (weightlifting), which is poorly served by existing wearable technology. Unlike running or cycling where heart rate correlates strongly with effort, resistance training involves intermittent high-intensity bursts separated by rest periods, making heart rate alone an insufficient metric. By combining ECG (cardiovascular effort), EMG (muscular effort), and accelerometry (movement volume), the system captures a multi-dimensional picture of workout quality that no single sensor could provide.

1.2 System Requirements

Functional Requirements:

- Real-time concurrent measurement of ECG, EMG, and accelerometer signals at 25 Hz sampling rate
- Automatic heart rate (BPM) computation using dynamic baseline beat detection
- Automatic repetition counting using a peak-valley state machine on accelerometer magnitude
- EMG muscle activation quantification with calibrated noise floor subtraction

- User selection menu that fetches registered users from the backend at startup
- Aggregate feature computation at workout completion (9 features: workout_id, duration_mins, avg_hr, max_hr, hr_spikes, pct_time_low, avg_emg, emg_fatigue, total_reps)
- Data transmission to AWS API Gateway upon workout completion
- AWS Lambda relay to FastAPI backend with automatic storage in Supabase PostgreSQL
- Web dashboard for reviewing workouts, selecting workout type, editing sensor data, and triggering AI analysis
- XGBoost-based effectiveness prediction with four classes (Low, Moderate, High, Maximum)
- SHAP-based per-prediction explanations identifying the top contributing factors
- AI Coach chatbot powered by Gemini 2.5 Flash with full user profile and workout context

Non-Functional Requirements:

- Thread-safe concurrent sensor reading using Python threading with I2C bus locking
- Coordinated I2C bus recovery mechanism to handle shared-bus contention errors
- Local JSON backup of all workout data as a failsafe
- JWT-based authentication via Supabase Auth for the web dashboard
- CORS-configured REST API for cross-origin frontend access
- Responsive web interface compatible with desktop and mobile browsers

1.3 Contributions

Toma Ionut-Adrian: Co-developed the hardware integration and prototype assembly. Architected and implemented the entire machine learning pipeline from scratch, including the synthetic dataset generation strategy designed to address specific physiological and hardware edge cases. Implemented the AI Coach integration, along with its associated backend logic and frontend interfaces. Authored the multi-threaded Python edge-processing script (`workout_tracker.py`) for the Raspberry Pi. Conducted primary research to formulate Proof of Concept solutions and authored the comprehensive scientific documentation.

Boros Fabian: Co-developed the hardware integration and sensor assembly. Architected and implemented the primary frontend web application and managed its deployment. Engineered the cloud data pipeline, specifically configuring the AWS API Gateway, authoring the AWS Lambda relay function, and establishing the connection to the backend. Led the practical hardware testing and calibration by wearing the prototype during physical trials.

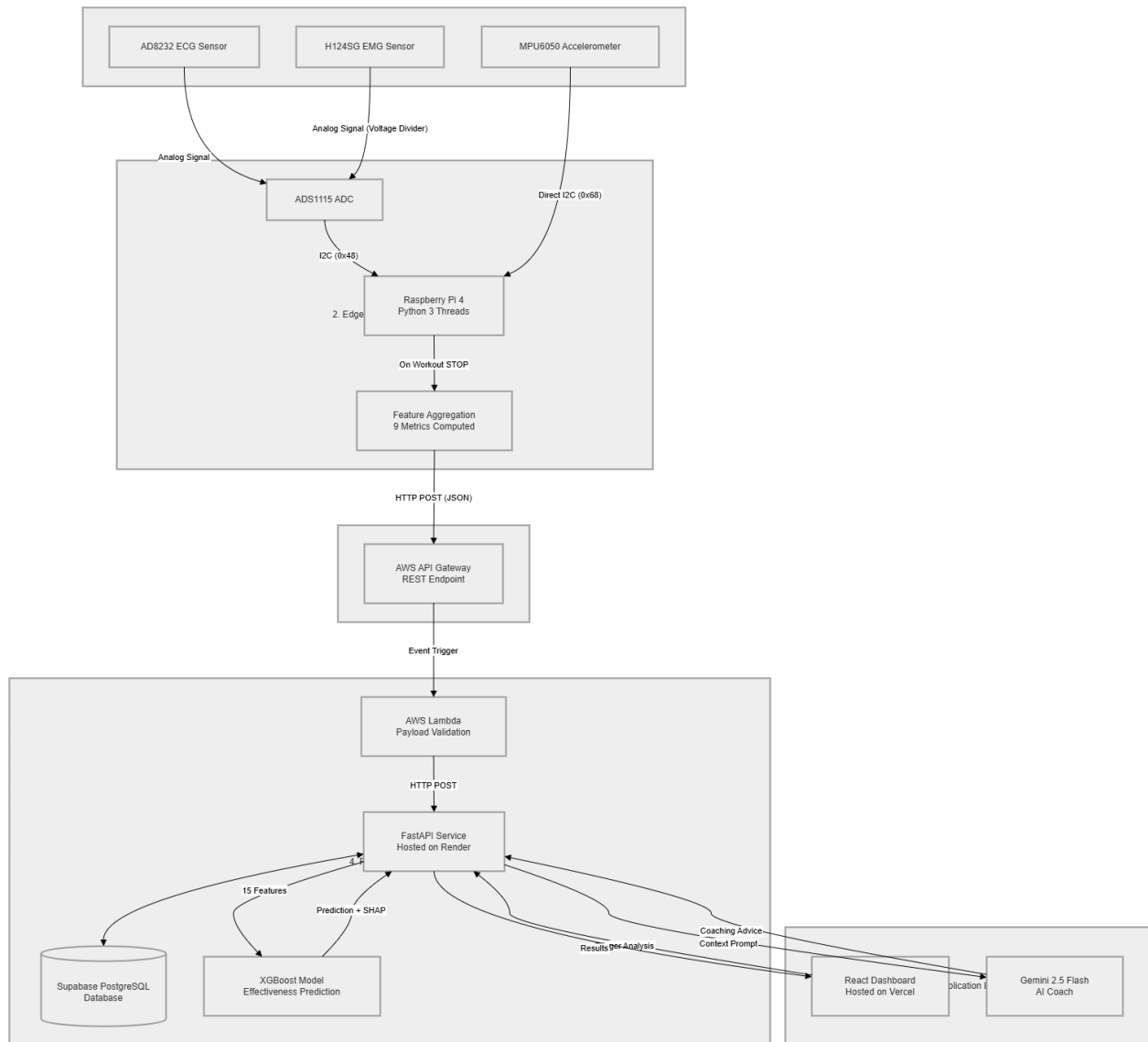
2. Design

2.1 System Architecture

The system follows a five-tier IoT architecture: **Perception Layer** (sensors), **Edge Processing Layer** (Raspberry Pi), **Network Layer** (HTTP/REST via AWS), **Processing Layer** (Lambda + FastAPI + XGBoost), and **Application Layer** (React dashboard + AI Coach).

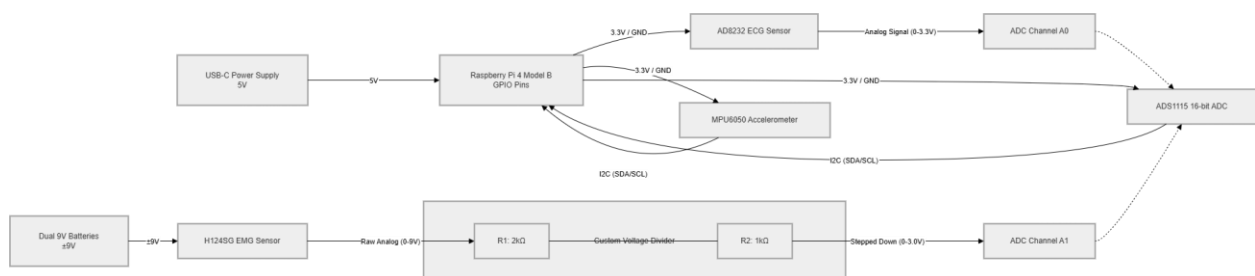
The end-to-end data flow is:

Sensors (ECG Sensor, EMG Sensor, MPU6050 Accelerometer) → [Analog signals via **ADS1115 ADC**; Digital via **Direct I2C**] → **Raspberry Pi 4** (Python, 3 concurrent threads) → **Feature Aggregation** (9 metrics computed on STOP) → (**HTTP POST**) → **AWS API Gateway** (eu-north-1) → **AWS Lambda** (validates + relays) → (**HTTP POST**) → **FastAPI on Render** → **Supabase PostgreSQL** → **React Dashboard on Vercel** → **User selects workout type** → **XGBoost Prediction + SHAP** → **AI Coach (Gemini 2.5 Flash)**



2.2 Hardware Architecture

The hardware consists of a Raspberry Pi 4 Model B (2GB RAM) as the central processing unit, connected to three sensor modules via a breadboard. The system captures three distinct classes of biological signals simultaneously.



zoom in required

Sensor Specifications

Component	Model	Signal Type	Interface	I2C Address	Supply Voltage	Output Range
ECG Monitor	AD8232	Analog	ADS1115 Ch. A0	-	3.3V	0-3.3V
EMG Sensor	H124SG	Analog	ADS1115 Ch. A1	-	+/-9V	0-9V (raw)
Accel/Gyro	MPU6050	Digital	Direct I2C	0x68	3.3V	+/-4g
ADC Converter	ADS1115	Digital	I2C Bus	0x48	3.3V	16-bit res
Microcomputer	Raspberry Pi 4	-	-	-	5V	-

Wiring and Connections

The AD8232 ECG sensor operates at 3.3V and outputs an analog voltage proportional to the detected cardiac electrical activity. Its output connects directly to channel A0 of the ADS1115 16-bit ADC, which communicates with the Raspberry Pi via the I2C bus (SDA/SCL pins). No voltage conversion is required as the AD8232 output stays within the 0–3.3V range of the ADS1115.

The EMG sensor presents a unique engineering challenge: it operates on a dual positive/negative 9V battery array, meaning its raw output signal can spike up to 9V — far exceeding the 3.3V maximum input of the ADS1115. To protect the system, the signal passes through a custom-built voltage divider circuit using a 2kΩ and 1kΩ resistor network, which steps down the maximum voltage from 9V to 3.0V before reaching channel A1 of the ADS1115. The voltage divider formula is:

$$V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2) = 9V \times 1k\Omega / (2k\Omega + 1k\Omega) = 3.0V$$

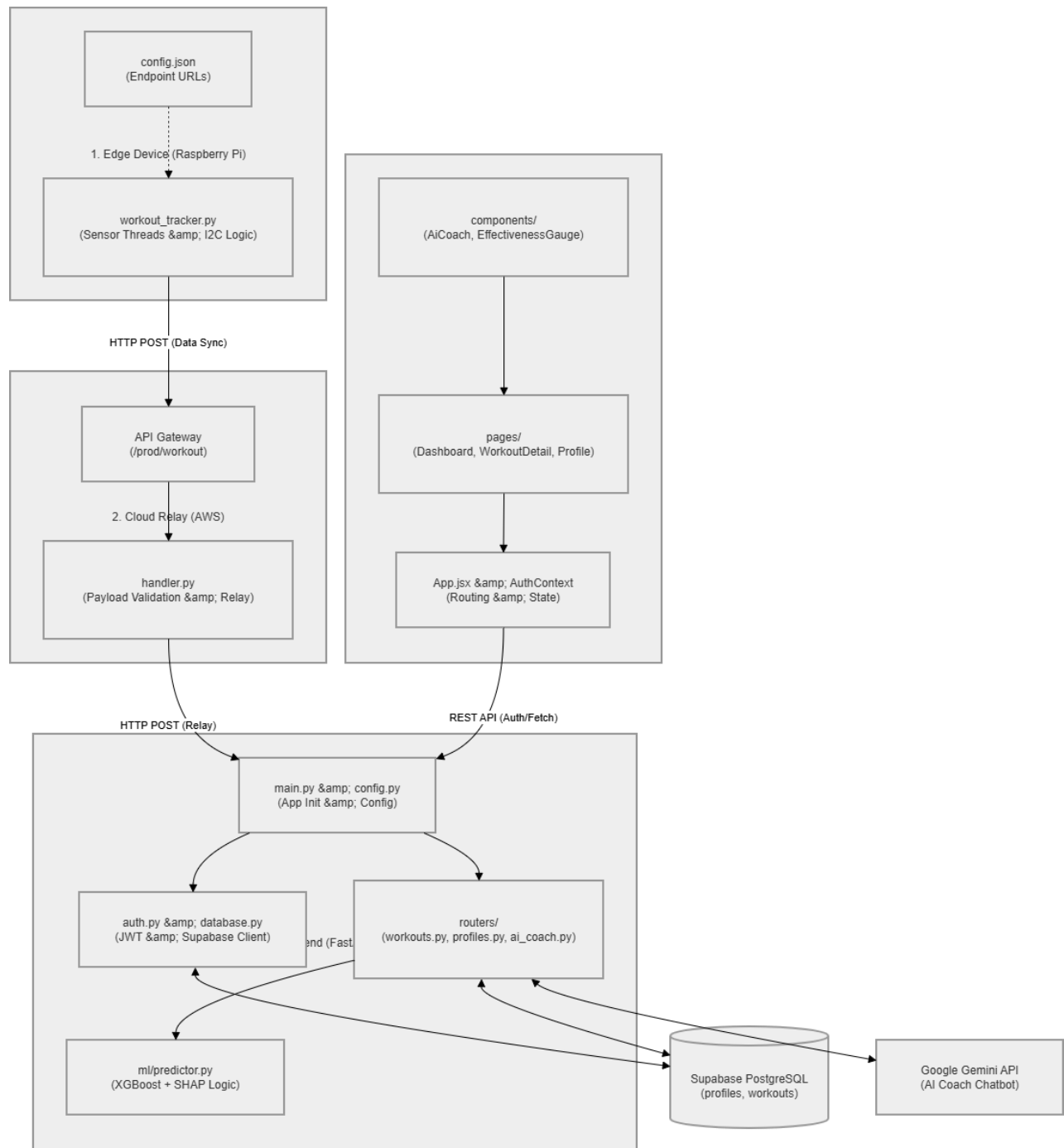
The MPU6050 6-axis accelerometer/gyroscope communicates directly via the I2C bus at address 0x68. It is configured for a ±4g full-scale range (sensitivity: 8192 LSB/g) using register 0x1C. No external ADC is required as the MPU6050 has its own internal 16-bit ADC.

Operational Range Metrics

Parameter	Value	Notes
I2C Bus Speed	100 kHz	Shared bus, default for busio.I2C
ADC Sampling Rate	250 SPS	ADS1115 max 860; 250 chosen for stability
Sensor Polling Rate	25 Hz (40ms)	Shared across all 3 sensor threads
Wi-Fi Range		Depends on environment
EMG Battery Life		Depends on battery capacity
Max Workout Time	Unlimited	Limited only by 9V EMG batteries

2.3 Software Architecture

The software architecture spans four environments: the edge device (Raspberry Pi), the cloud relay (AWS), the web backend (FastAPI on Render), and the web frontend (React on Vercel).



Edge Device (Raspberry Pi)

The Raspberry Pi runs a Python 3 script (`workout_tracker.py`, 843 lines) that manages three concurrent sensor threads, each reading its respective sensor at 25 Hz. The application follows a multi-threaded producer-consumer pattern with shared data stores protected by `threading.Lock`. A second lock (`i2c_lock`) serializes access to the shared I2C bus, and a coordinated bus recovery mechanism pauses all threads simultaneously when consecutive I2C errors exceed a threshold.

Key source files:

File	Lines	Purpose
`workout_tracker.py`	843	Main application: sensor threads, feature computation, AWS transmission
`config.json`	5	AWS API Gateway URL and backend URL configuration

Cloud (AWS)

Two AWS services handle the data relay:

AWS Service	Region	Purpose	Pricing Tier
API Gateway	eu-north-1	REST endpoint receiving POST requests from the Pi	Free tier
Lambda	eu-north-1	Validates payload, forwards to FastAPI backend	Free tier

The API Gateway exposes a single POST endpoint at `/prod/workout`. When invoked, it triggers the Lambda function (`handler.py`, 91 lines), which: (1) parses and validates the JSON body for all 9 required fields; (2) forwards the payload to the FastAPI backend via HTTP POST; (3) returns the backend's response (including the assigned database ID) to the caller.

Backend (FastAPI on Render)

The backend is a Python FastAPI application deployed on Render as a web service. It exposes REST API endpoints for user profiles, workout data management, AI analysis, and an AI Coach chatbot.

Key source files:

File	Lines	Purpose
`main.py`	60	App initialization, CORS, router registration
`config.py`	27	Environment variable loading (Supabase, Gemini, model paths)
`database.py`	21	Supabase client initialization (anon + admin)
`auth.py`	42	JWT token verification middleware
`routers/profiles.py`	145	User profile CRUD operations
`routers/workouts.py`	354	Workout data ingestion, management, AI analysis trigger
`routers/ai_coach.py`	252	Gemini-powered fitness chatbot
`ml/predictor.py`	206	XGBoost model loading, prediction, SHAP explanation

REST API Endpoints:

Method	Endpoint	Auth	Description
GET	`/`	No	Health check
GET	`/health`	No	Health status
GET	`/profiles/list`	No	List all users (for Pi user selection)
GET	`/profiles/me`	JWT	Get current user's profile
POST	`/profiles/me`	JWT	Create profile with demographics
PUT	`/profiles/me`	JWT	Update demographic data
POST	`/workouts`	No	Receive sensor data from Pi/Lambda
GET	`/workouts`	JWT	List user's workouts
GET	`/workouts/{id}`	JWT	Get single workout details
PUT	`/workouts/{id}/type`	JWT	Set workout type (HILV/LIHW/hypertr ophy/endurance_lifting)
PUT	`/workouts/{id}/reps`	JWT	Manually set rep count
PUT	`/workouts/{id}/data`	JWT	Edit any sensor data fields
POST	`/workouts/{id}/analyze`	JWT	Run XGBoost prediction + SHAP
POST	`/ai-coach/chat`	JWT	Send message to Gemini AI Coach

Frontend (React on Vercel)

The frontend is a single-page application built with React 19, Vite 8, React Router 7, Framer Motion, and Recharts. It is deployed on Vercel at `fitness-monitoring-system.vercel.app`.

Key source files:

File	Purpose
`App.jsx`	Root component with routing (Landing, Login, Register, Dashboard, WorkoutDetail, Profile)
`pages/Dashboard.jsx`	Workout list with cards, manual workout form
`pages/WorkoutDetail.jsx`	Detailed view with sensor data, effectiveness gauge, SHAP explanation, AI Coach
`pages/Landing.jsx`	Public landing page
`pages/Login.jsx` / `Register.jsx`	Supabase authentication
`components/AiCoach.jsx`	Gemini chatbot interface
`components/EffectivenessGauge.jsx`	Visual effectiveness rating display
`components/ShapExplanation.jsx`	SHAP factor visualization
`components/WorkoutCard.jsx`	Workout summary card
`components/DemographicForm.jsx`	User profile form
`context/AuthContext.jsx`	Supabase auth state management

Database Schema

The system uses Supabase-managed PostgreSQL with two tables:

Table: profiles

Column	Type	Constraints	Description
id	UUID	PK, FK → auth.users(id) ON DELETE CASCADE	Supabase Auth user ID
first_name	VARCHAR(100)	Nullable	User's first name
last_name	VARCHAR(100)	Nullable	User's last name
age	INTEGER	NOT NULL	User's age (13-100)
weight_kg	NUMERIC(4,1)	NOT NULL, DEFAULT 80.0	Body weight in kg
fitness_level	VARCHAR(10)	NOT NULL, CHECK IN ('low','medium','high')	Self-reported fitness level
athlete_type	VARCHAR(20)	NOT NULL, CHECK IN ('powerlifter','hybrid','gym_bro','non_athletic')	Training style
body_fat_pct	NUMERIC(4,1)	NOT NULL, CHECK 3.0-50.0	Body fat percentage
limb_length	VARCHAR(10)	NOT NULL, CHECK IN ('short','medium','long')	Relative limb length
created_at	TIMESTAMPTZ	DEFAULT now()	Profile creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Last update timestamp

Table: workouts

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Auto-generated workout ID
user_id	UUID	NOT NULL, FK → profiles(id) ON DELETE CASCADE	Owner user
workout_type	VARCHAR(20)	CHECK IN ('HILV','LIHV','hypertrophy','endurance_lifting')	Training style (set by user post-workout)
duration_mins	NUMERIC(5,1)	NOT NULL	Workout duration in minutes
avg_hr	NUMERIC(5,1)	NOT NULL	Average heart rate (BPM)
max_hr	NUMERIC(5,1)	NOT NULL	Maximum heart rate (BPM)
hr_spikes	INTEGER	NOT NULL, DEFAULT 0	Count of HR jumps > threshold
pct_time_low	NUMERIC(5,1)	NOT NULL, DEFAULT 0	% time in low HR zone (<100 BPM)
avg_emg	NUMERIC(6,1)	NOT NULL, DEFAULT 0	Average EMG activation (0-1000)
emg_fatigue	NUMERIC(5,1)	NOT NULL, DEFAULT 0	EMG fatigue index (0-60%)
total_reps	INTEGER	NOT NULL, DEFAULT 0	Repetitions detected by MPU6050
effectiveness_label	INTEGER	Nullable	AI prediction (0-3)
effectiveness_name	VARCHAR(10)	Nullable	Human label (Low/Moderate/High/Maximum)
confidence	NUMERIC(5,4)	Nullable	Prediction confidence (0-1)
probabilities	JSONB	Nullable	Per-class probability distribution
explanation	TEXT	Nullable	Natural language SHAP explanation
top_factors	JSONB	Nullable	Top SHAP contributing factors
status	VARCHAR(20)	DEFAULT 'pending', CHECK IN ('pending','analyzed')	Workflow status
recorded_at	TIMESTAMPTZ	DEFAULT now()	Recording timestamp

Indexes:

Index Name	Column(s)	Type	Purpose
idx_workouts_user_id	user_id	B-tree	Fast lookup by user
idx_workouts_recorded_at	recorded_at DESC	B-tree	Chronological ordering

2.4 AI / ML Architecture

The AI component uses an XGBoost (eXtreme Gradient Boosting) multi-class classifier to predict workout effectiveness on a four-tier scale. XGBoost was chosen over neural network alternatives because: (1) it is best-in-class for tabular data with non-linear conditional rules; (2) it natively handles the conditional thresholds that define workout quality (e.g., “IF high intensity AND short duration THEN Low”); (3) TreeSHAP provides exact, polynomial-time per-prediction explanations; (4) no feature scaling is required; (5) it trains in seconds on the dataset size (~51,000 samples). Importantly, deploying a complex neural network for a constrained 15-feature tabular dataset would constitute significant overengineering for this specific use case.

Model Architecture

Parameter	Value
Algorithm	XGBoost (`multi:softprob`)
Classes	4 (Low=0, Moderate=1, High=2, Maximum=3)
Features	15 (7 demographic + 8 sensor)
Categorical Features	4 (fitness_level, workout_type, athlete_type, limb_length) — handled natively
Tree Method	`hist` (histogram-based, fast)
Hyperparameter Tuning	RandomizedSearchCV (50 iterations, 5-fold stratified CV)
Best n_estimators	500
Best max_depth	6
Best learning_rate	0.2115
Test Accuracy	96.12%
Test F1 (macro)	96.16%

Feature Engineering

The model uses 15 input features divided into two categories:

Demographic Features (7) — from the user’s profile, providing biological context:

Feature	Type	Range	Physiological Rationale
age	Integer	16-65	Limits max HR (~220-age); normalizes effort relative to biological ceiling
weight_kg	Float	50-130 kg	Higher body mass increases cardiovascular load for identical exercises
fitness_level	Categorical	low/medium/high	Trained athletes have higher stroke volume → lower avg_hr for same work
athlete_type	Categorical	powerlifter/hybrid/gym_bro/non_athletic	Behavioral context (powerlifters have low HR + extreme EMG)
body_fat_pct	Float	3-50%	Affects cardiovascular load independent of muscle mass
limb_length	Categorical	short/medium/long	Longer limbs = more mechanical work per rep = earlier fatigue
workout_type	Categorical	HIILV/LIHV/hypertrophy/endurance_lifting	Defines expected volume/intensity profile

Sensor Features (8) — from the Raspberry Pi hardware:

Feature	Type	Range	Source Sensor
duration_mins	Float	1-90+ min	System clock (workout_start to workout_end)
avg_hr	Float	50-200 BPM	AD8232 ECG via ADS1115
max_hr	Float	50-210 BPM	AD8232 ECG via ADS1115
hr_spikes	Integer	0-15+	AD8232 ECG (consecutive BPM jumps > 70)
pct_time_low	Float	0-100%	AD8232 ECG (% readings < 100 BPM)
avg_emg	Float	0-1000	EMG sensor via ADS1115
emg_fatigue	Float	0-60%	EMG sensor (amplitude drop: first vs last quarter)
total_reps	Integer	0-300+	MPU6050 accelerometer (peak-valley detection)

Explainability: SHAP (SHapley Additive exPlanations)

TreeSHAP is used to decompose each prediction into per-feature contributions. For every workout analysis, the system identifies the top 3 features that most influenced the predicted effectiveness label and generates a natural-language explanation. Example output:

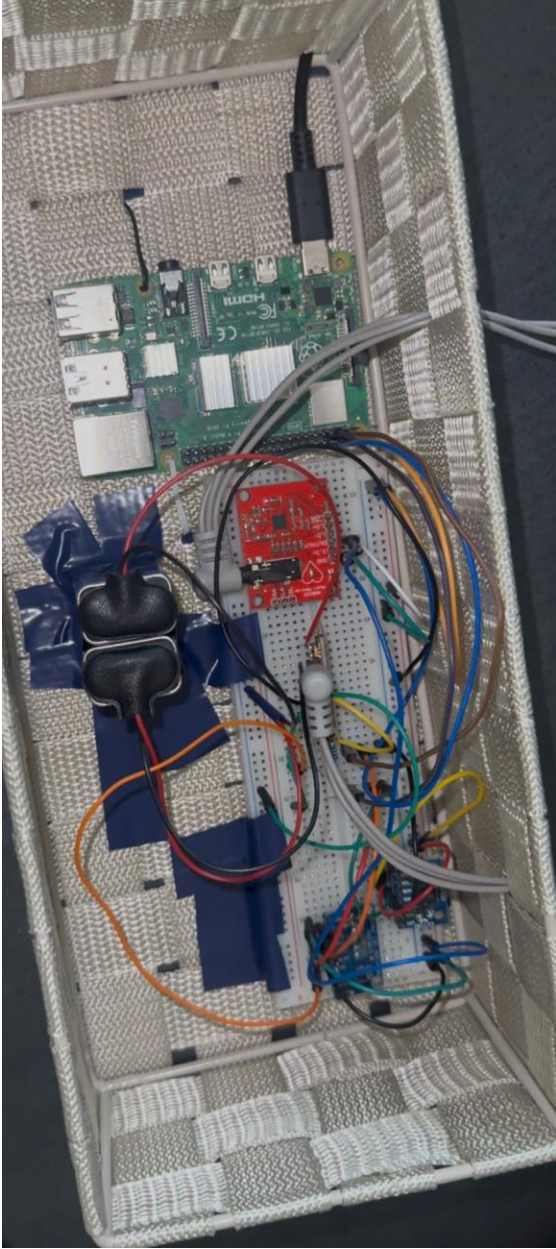
“High effectiveness (confidence: 87%). Key factors: strong muscle engagement (EMG: 480); long session duration (45 min); elevated average heart rate (135 BPM).”

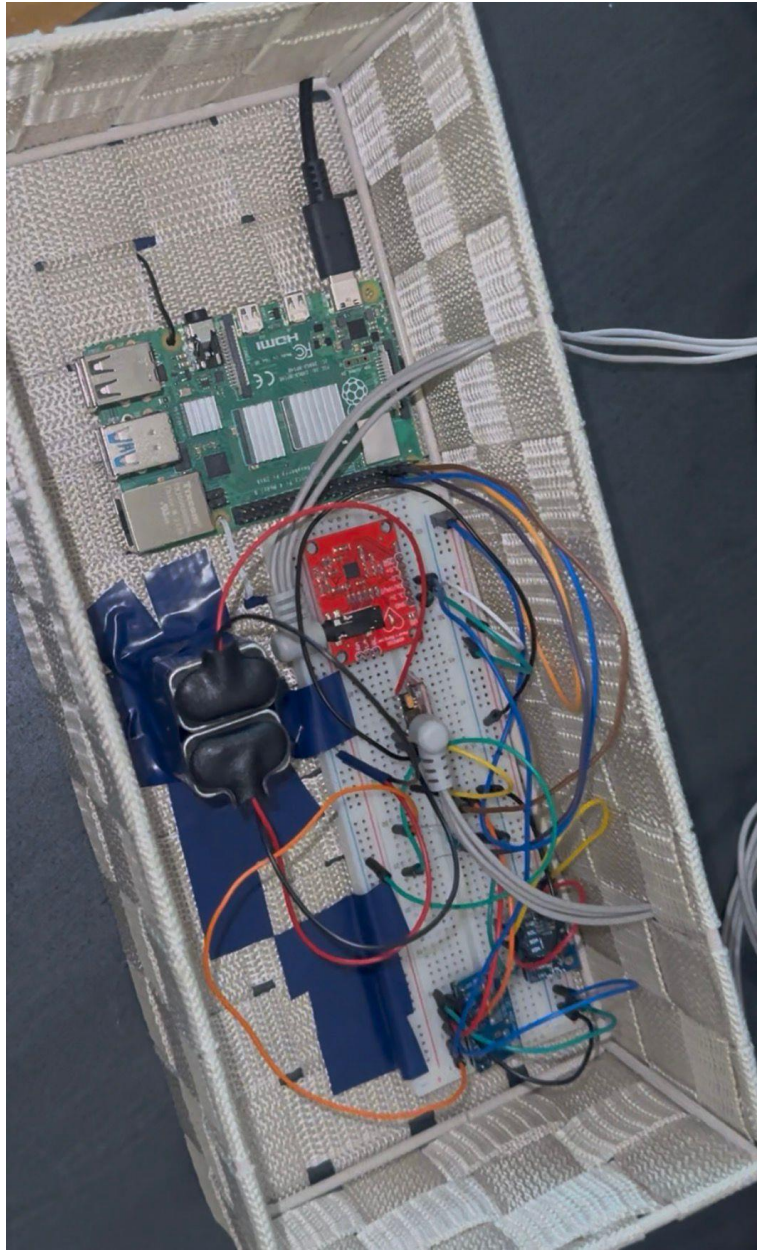
Effectiveness Labels:

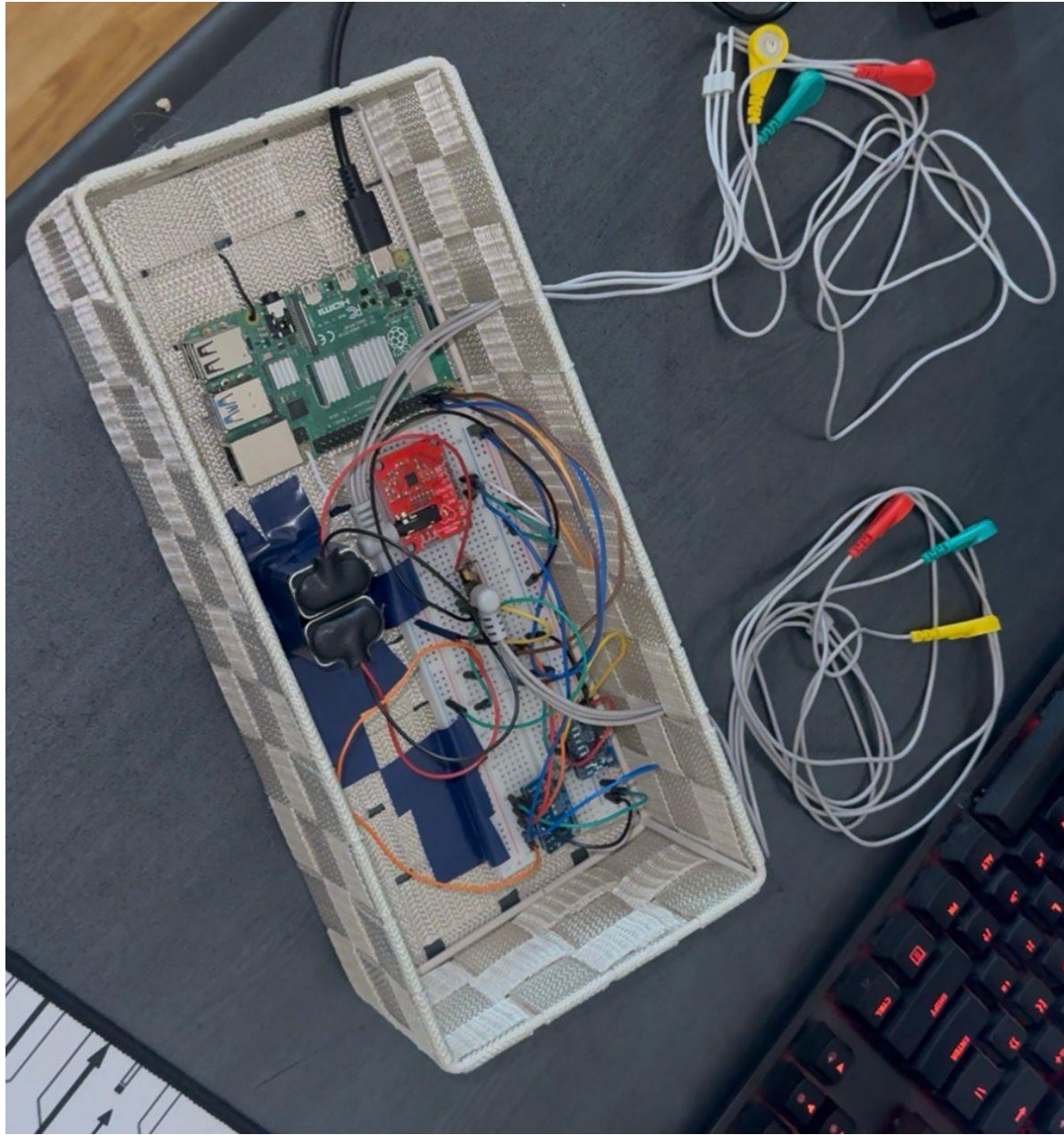
Label	Name	Description
0	Low	Too short, phone scrolling, going through the motions, warmup only
1	Moderate	Light recovery, average session, short but decent, beginner cautious
2	High	Solid structured session, intense shorter workout, superset style
3	Maximum	Beast mode, heavy powerlifting, high-rep endurance, competition training

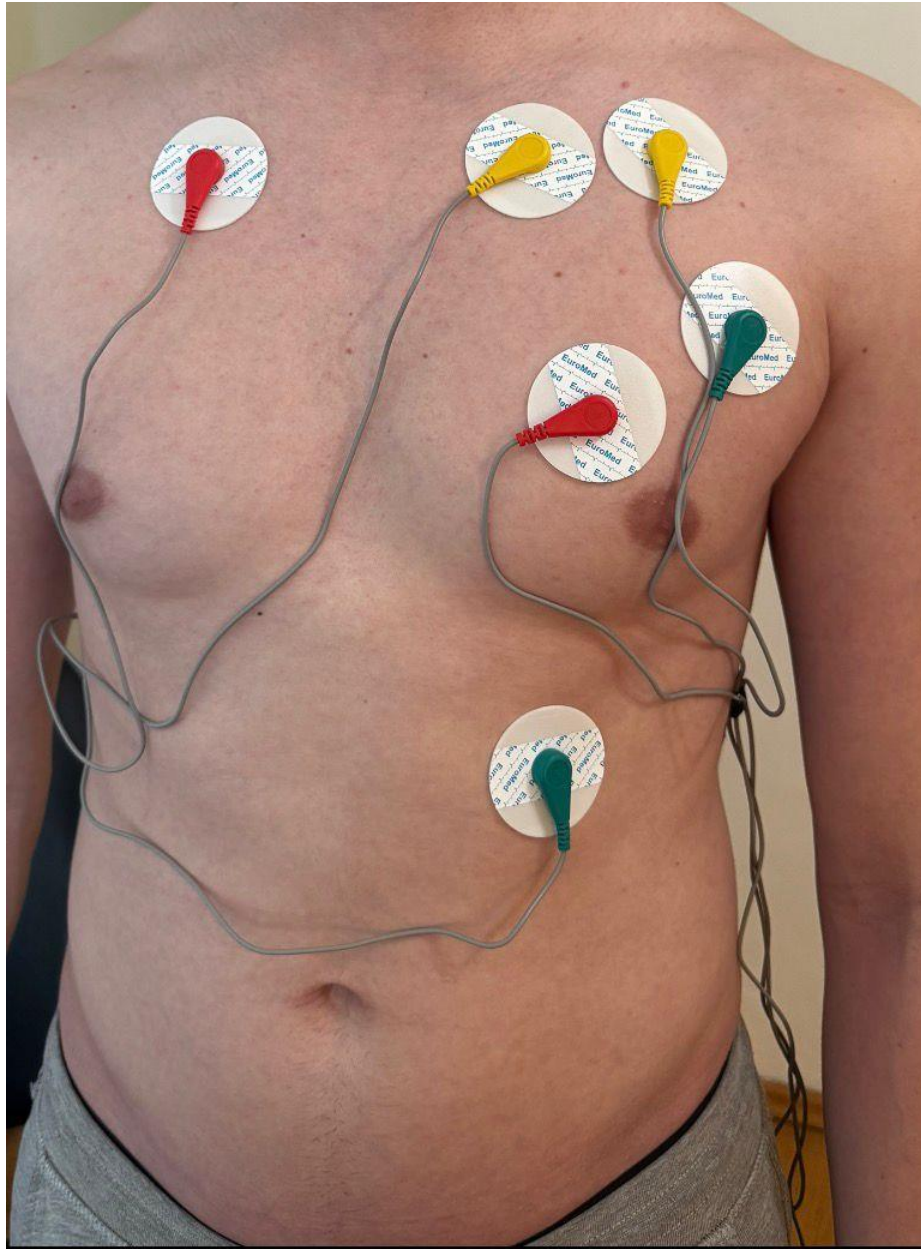
3. Implementation

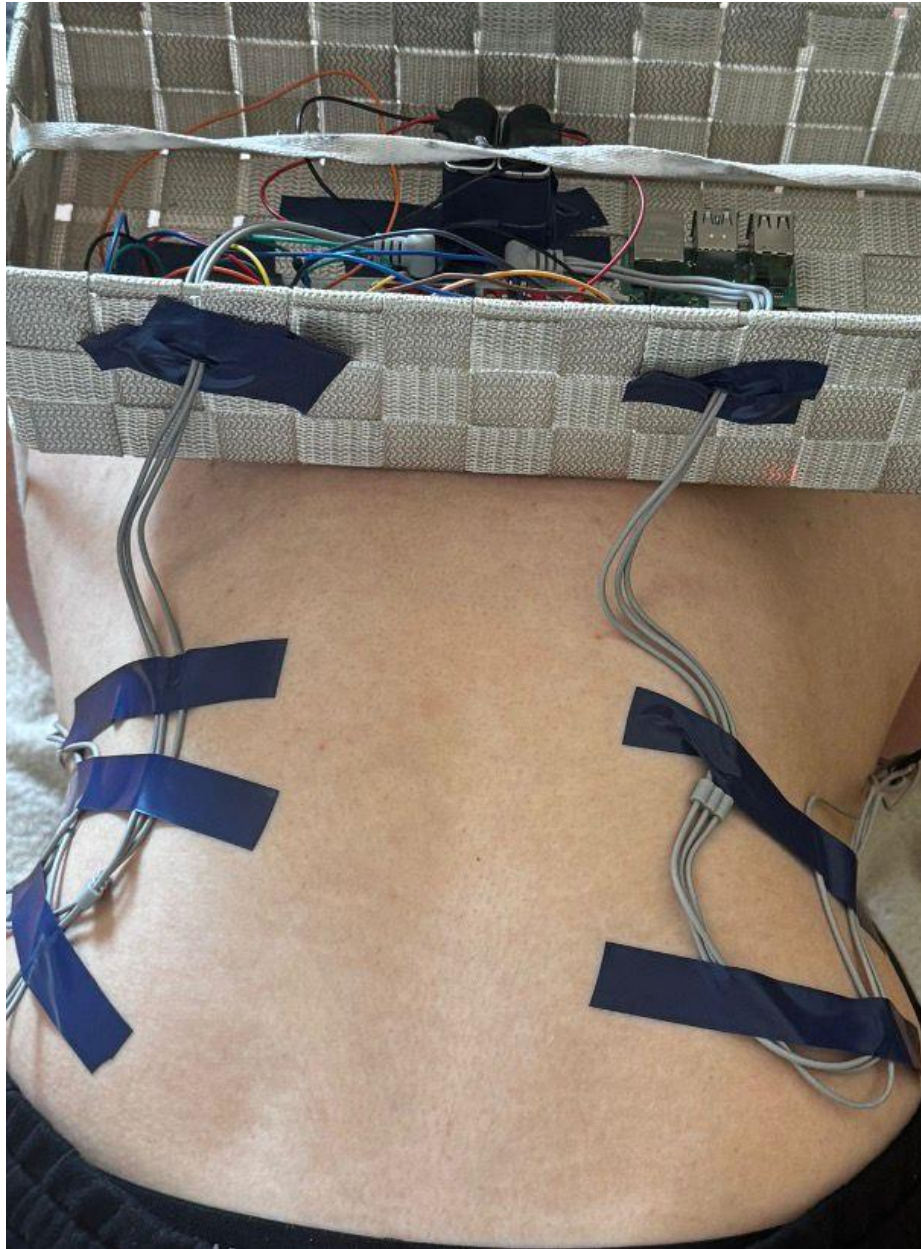
3.1 Hardware Setup and Sensor Integration











Hardware Layout:

Hardware Layout

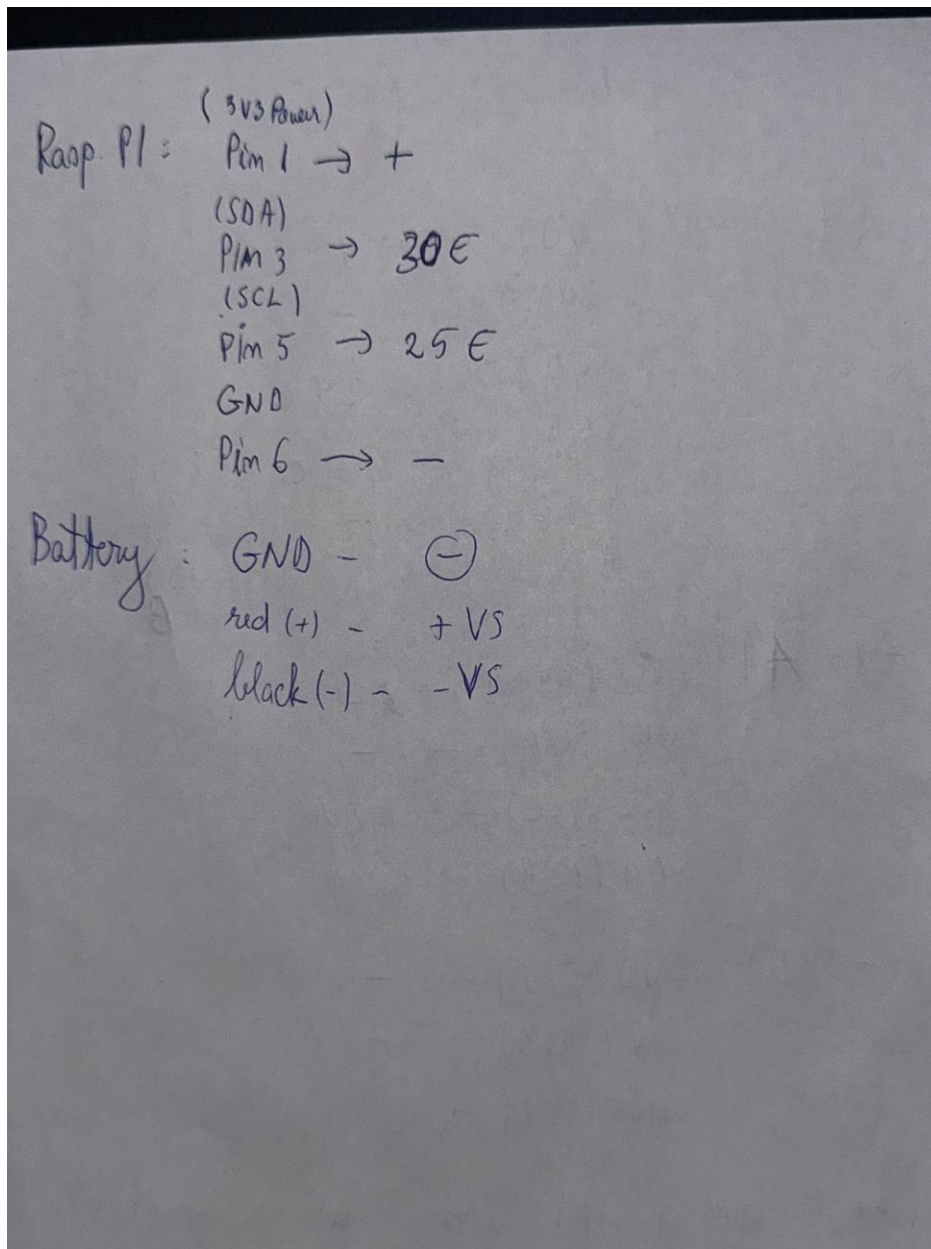
AD1015 (1-10 H) VDD (10j) $\rightarrow +$
GND (9i) $\rightarrow -$
SCL (8j) $\rightarrow 25C$
SDA (7i) $\rightarrow 30C$
ADDR (6j) $\rightarrow -$
AO (4j) $\rightarrow$ OUTPUT (55B) (AD8232)

MPU (8-15 A) VCC (15C) $\rightarrow +$
GND (14B) $\rightarrow -$
SCL (13C) $\rightarrow 25A$
SDA (12B) $\rightarrow 30A$

AD8232 (53-58D) GND (53B) $\rightarrow -$
3.3V (54A) $\rightarrow +$
OUTPUT (55B) $\rightarrow$ AO (4j)

Sensor Hunchi (38-39F; 41-43F) SIG (39G) $\rightarrow 33G$
GND (38G) $\rightarrow -$

Voltage Divider $\frac{1}{2}$: SIG (39G) $\rightarrow 33G$
resistora 2K $\rightarrow 33H \rightarrow 29J$
resistora 1K $\rightarrow 29I \rightarrow 24F$
cablu 29J $\rightarrow A1$ (AD1015) 3J
cablu 24J $\rightarrow -$



Mixed-Voltage Safety

The most critical hardware engineering challenge was safely combining the 9V EMG sensor with the 3.3V digital ecosystem. The custom voltage divider ($2k\Omega + 1k\Omega$) guarantees that even if the EMG sensor outputs its maximum 9V, the ADS1115 input never exceeds 3.0V. This was verified with a multimeter before connecting to the ADC.

I2C Bus Configuration

All digital communication uses the I2C protocol on the Raspberry Pi's default SDA/SCL pins. Two devices share the bus:

Device	I2C Address	Data Rate	Bus Usage
ADS1115 (ADC)	0x48	250 SPS (configured)	ECG + EMG analog reads
MPU6050 (Accel)	0x68	25 Hz polling	Accelerometer reads

The shared bus creates contention risks. This is mitigated in software via `i2c_lock` (see Section 3.2).

3.2 Edge Device Software (Raspberry Pi)

The main script (`workout_tracker.py`, 843 lines) manages the entire workout lifecycle. The application runs an interactive loop: select user → start workout → run sensor threads → press ENTER to stop → compute features → send to AWS → save locally.

Multi-Threaded Sensor Architecture

Three daemon threads run concurrently during a workout, each reading its respective sensor at 25 Hz (40ms intervals):

Thread	Function	Sensor	Data Produced	Key Algorithm
'ecg_thread'	Heart rate detection	AD8232 via ADS1115 A0	'ecg_bpm_log[]' — list of (timestamp, BPM)	Dynamic baseline beat detection
'emg_thread'	Muscle activation	EMG via ADS1115 A1	'emg_signal_log[]' — list of (timestamp, activation)	High-pass filter + noise subtraction
'mpu_thread'	Rep counting	MPU6050 direct I2C	'rep_timestamps[]' — list of timestamps	Peak-valley state machine

Thread Synchronization

Two locks coordinate access:

1. **data_lock** (`threading.Lock`): Protects the three shared data lists (`ecg_bpm_log`, `emg_signal_log`, `rep_timestamps`) from concurrent read/write corruption.
2. **i2c_lock** (`threading.Lock`): Serializes all I2C bus access. Every sensor read acquires this lock before touching the bus, ensuring only one device communicates at a time.

Coordinated I2C Bus Recovery

When I2C errors cascade (3+ consecutive errors from any thread), a global recovery mechanism activates:

```
i2c_recovering = True
i2c_recover_until = time.time() + 0.6 # 600ms pause
```


All three threads check this flag at the top of their loop and back off simultaneously, giving the I2C bus time to reset. This prevents the common failure mode where one thread's error recovery attempt collides with another thread's read attempt.

3.3 Core Algorithms

Algorithm 1: ECG Beat Detection (Dynamic Baseline Threshold)

The ECG beat detection uses a moving-average baseline that slowly adapts to breathing-induced drift, combined with a fixed offset threshold above the baseline to detect R-wave peaks:

1. Initialize `moving_baseline` = first voltage reading
2. For each sample:
 - a. $\text{moving_baseline} = \text{baseline} \times 0.99 + \text{voltage} \times 0.01$ (slow adaptation)
 - b. $\text{dynamic_threshold} = \text{moving_baseline} + 0.06V$ (fixed offset)
 - c. IF $\text{voltage} > \text{dynamic_threshold}$ AND cooldown expired (0.5s):
 - Calculate $\text{BPM} = 60 / \text{interval_since_last_beat}$
 - IF $40 \leq \text{BPM} \leq 200$: log (timestamp, BPM)
 - Update `last_beat_time`

The 0.99 smoothing factor means the baseline tracks breathing and posture drift but cannot follow the rapid R-wave peaks, ensuring they cross the threshold.

Algorithm 2: EMG Activation Quantification

The EMG signal processing uses a high-pass filter (baseline subtraction) followed by calibrated noise floor removal:

1. CALIBRATION (2 seconds, arm relaxed):
 - Record min/max voltage $\rightarrow \text{noise_barrier} = (\text{max} - \text{min}) / 2 + 0.015V$
 - Clamp $\text{noise_barrier} \leq 0.145V$
2. For each sample:
 - a. $\text{baseline} = \text{baseline} \times 0.99 + \text{voltage} \times 0.01$
 - b. $\text{raw_deviation} = |\text{voltage} - \text{baseline}|$
 - c. $\text{active_signal} = \max(0, \text{raw_deviation} - \text{noise_barrier})$
 - d. $\text{activation} = \min(\text{active_signal} \times 5000, 1000)$
 - e. Log (timestamp, activation)

Algorithm 3: MPU6050 Rep Counting (Peak-Valley State Machine)

The rep counter uses a two-state machine that detects the full up-down cycle of a repetition:

1. CALIBRATION (1 second, hold still):
 - $\text{rest_magnitude} = \text{mean}(\text{acceleration magnitude readings})$ (~1.0g at rest)
2. Smooth raw magnitude with EMA: $\text{smoothed} = 0.5 \times \text{raw} + 0.5 \times \text{previous}$
3. $\text{deviation} = |\text{smoothed} - \text{rest_magnitude}|$
4. State machine:
 - IDLE: IF $\text{deviation} > 0.2g \rightarrow$ transition to PEAK
 - PEAK: track peak deviation

IF deviation < 0.1g (return to rest) AND cooldown (1.0s) expired:
→ COUNT REP, transition to IDLE

Feature Computation (On Workout STOP)

When the user presses ENTER, all threads stop and 9 features are computed:

Feature	Computation
workout_id	UUID hex (8 chars)
duration_mins	(workout_end - workout_start) / 60
avg_hr	mean(all BPM readings)
max_hr	max(all BPM readings)
hr_spikes	count of consecutive BPM jumps > 70
pct_time_low	(readings < 100 BPM) / total × 100
avg_emg	mean(all activation values)
emg_fatigue	(mean_first_quarter - mean_last_quarter) / mean_first_quarter × 100, clamped to 60
total_reps	len(rep_timestamps)

3.4 Cloud Infrastructure (AWS)

All AWS services operate within the free tier:

Service	Resource	Free Tier Limit	Project Usage
API Gateway	REST API	1M requests/month	~100 requests/month
Lambda	Function invocations	1M requests/month	~100 invocations/month
Lambda	Compute	400,000 GB-seconds/month	Negligible

The Lambda function (`handler.py`, 91 lines) performs three tasks: 1. Parses the JSON body from API Gateway 2. Validates all 9 required fields are present 3. Forwards the payload via HTTP POST to `{BACKEND_URL}/workouts`

Communication Protocols

Segment	Protocol	Port	Security	Payload Format
PI → API Gateway	HTTPS (REST)	443	TLS 1.2+	JSON
API Gateway → Lambda	AWS internal	—	IAM	JSON event
Lambda → Backend	HTTPS (REST)	443	TLS 1.2+	JSON
Frontend → Backend	HTTPS (REST)	443	TLS 1.2+ + JWT Bearer	JSON
Backend → Supabase	HTTPS (REST)	443	API Key + Service Role	Supabase SDK
Backend → Gemini	HTTPS (REST)	443	API Key	JSON

3.5 Backend (FastAPI on Render)

The FastAPI backend handles four responsibilities: user management, workout data persistence, AI analysis orchestration, and AI Coach proxying.

Workout Data Flow:

POST `/workouts` (from Lambda)

- Verify `user_id` exists in `profiles` table
- Insert workout row with `status='pending'`
- Return created workout (including generated UUID)

AI Analysis Flow:

POST `/workouts/{id}/analyze` (from frontend, JWT auth)

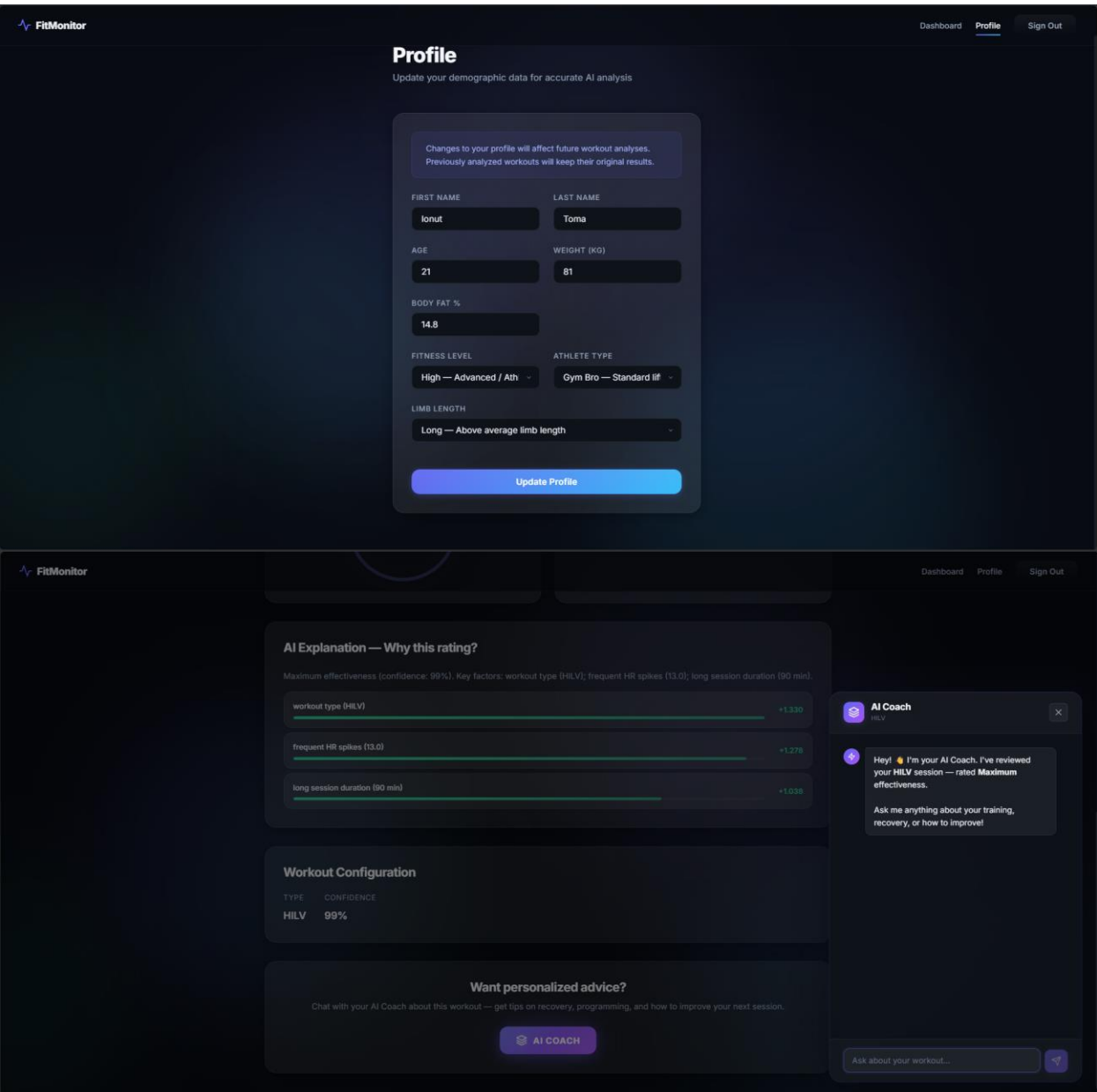
- Fetch workout data from database
- Verify `workout_type` is set

- Fetch user demographics from profiles table
- Merge 7 demographic + 8 sensor features into 15-feature vector
- Pass to XGBoost model → predict class + probabilities
- Compute SHAP values → extract top 3 contributing factors
- Generate natural-language explanation
- Update workout row with prediction results, set status='analyzed'

3.6 Frontend (React on Vercel)

The frontend is a single-page application with the following pages:

Page	Route	Auth	Description
Landing	`/`	No	Public landing with feature overview
Login	`/login`	No	Supabase email/password auth
Register	`/register`	No	Account creation with demographic form
Dashboard	`/dashboard`	JWT	Workout list with cards, manual workout form
Workout Detail	`/workout/:id`	JWT	Full workout view, type selection, AI analysis, AI Coach
Profile	`/profile`	JWT	Edit demographic data



Workout Details

MAXIMUM

Wednesday, June 10, 2026 at 07:14 PM

Sensor Data

90

DURATION

115

AVG HR

165

MAX HR

13

HR SPIRES

30

LOW ZONE

450

AVG EMG

18

FATIGUE

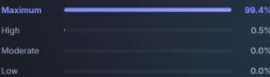
121

TOTAL REPS

Effectiveness Rating

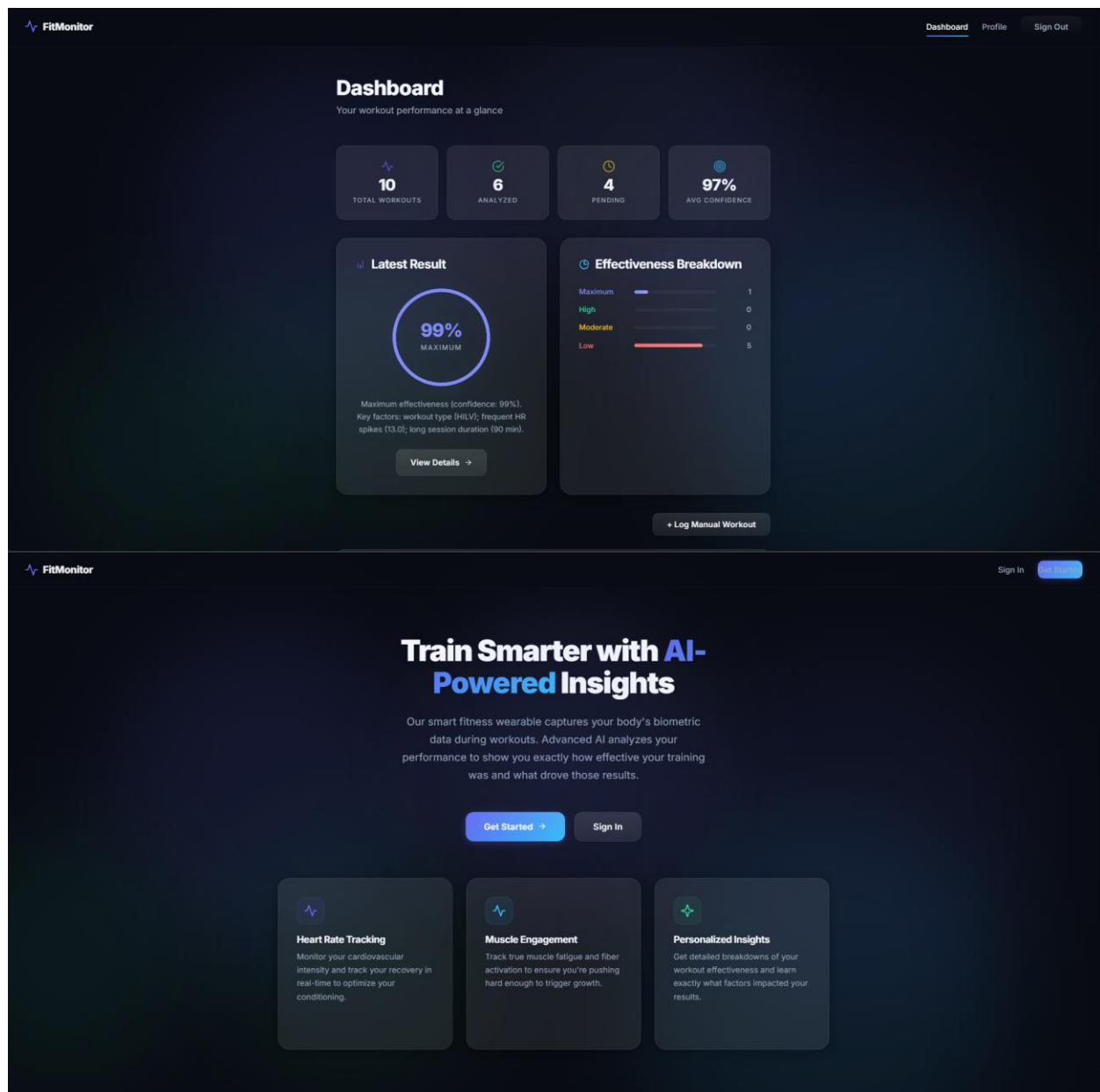


Class Probabilities



+ Log Manual Workout

Jun 10 07:14 PM	High Intensity, Low Volume 90 min · 121 reps · Avg HR 115	MAXIMUM	99% CONFIDENCE
Jun 10 07:05 PM	Low Intensity, High Volume 0.5 min · 19 reps · Avg HR 102.7	LOW	100% CONFIDENCE
Jun 10 06:58 PM	High Intensity, Low Volume 0.4 min · 8 reps · Avg HR 102.1	LOW	90% CONFIDENCE
Jun 8 12:23 PM	High Intensity, Low Volume 10 min · 20 reps · Avg HR 120	LOW	94% CONFIDENCE
Jun 5 06:51 PM	Hypertrophy 1.4 min · 21 reps · Avg HR 101.3	LOW	100% CONFIDENCE
Jun 5 06:48 PM	Type not set 0.6 min · 1 reps · Avg HR 101.7	SELECT TYPE	>
Jun 5 06:16 PM	Type not set 0.1 min · 0 reps · Avg HR 100	SELECT TYPE	>
Jun 5	Type not set	SELECT TYPE	>



3.7 Machine Learning Pipeline

Training Data Generation

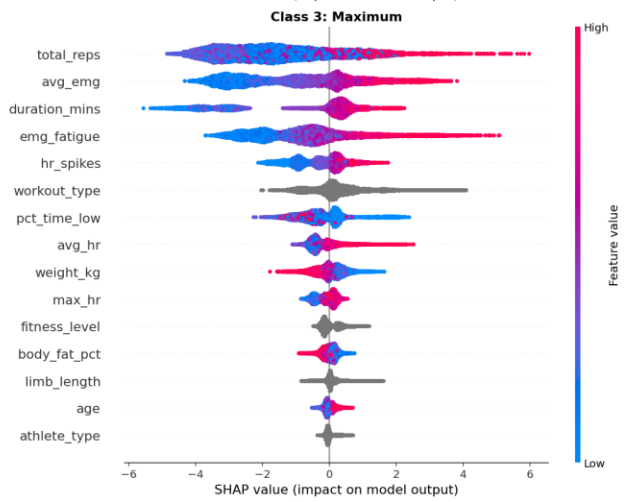
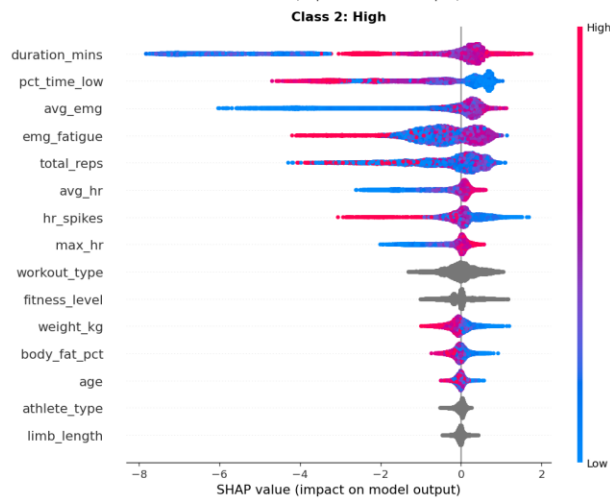
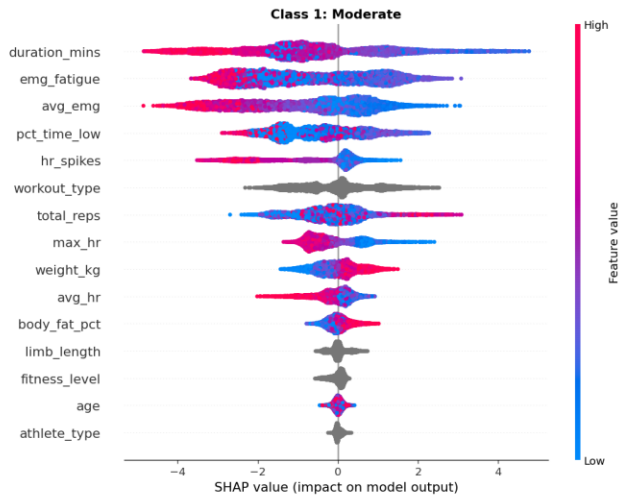
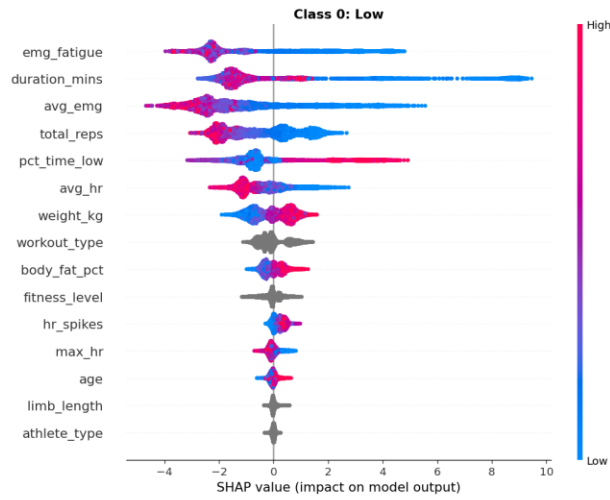
The training dataset consists of approximately 51,000 synthetic workout samples generated by `generate_synthetic_data.py` (1,002 lines). The generator defines 29 sub-profiles across 4 effectiveness classes, each specifying realistic ranges for all sensor features. Additionally, 8 edge-case profiles use linear interpolation between boundary sub-profiles to teach the model ambiguous scenarios.

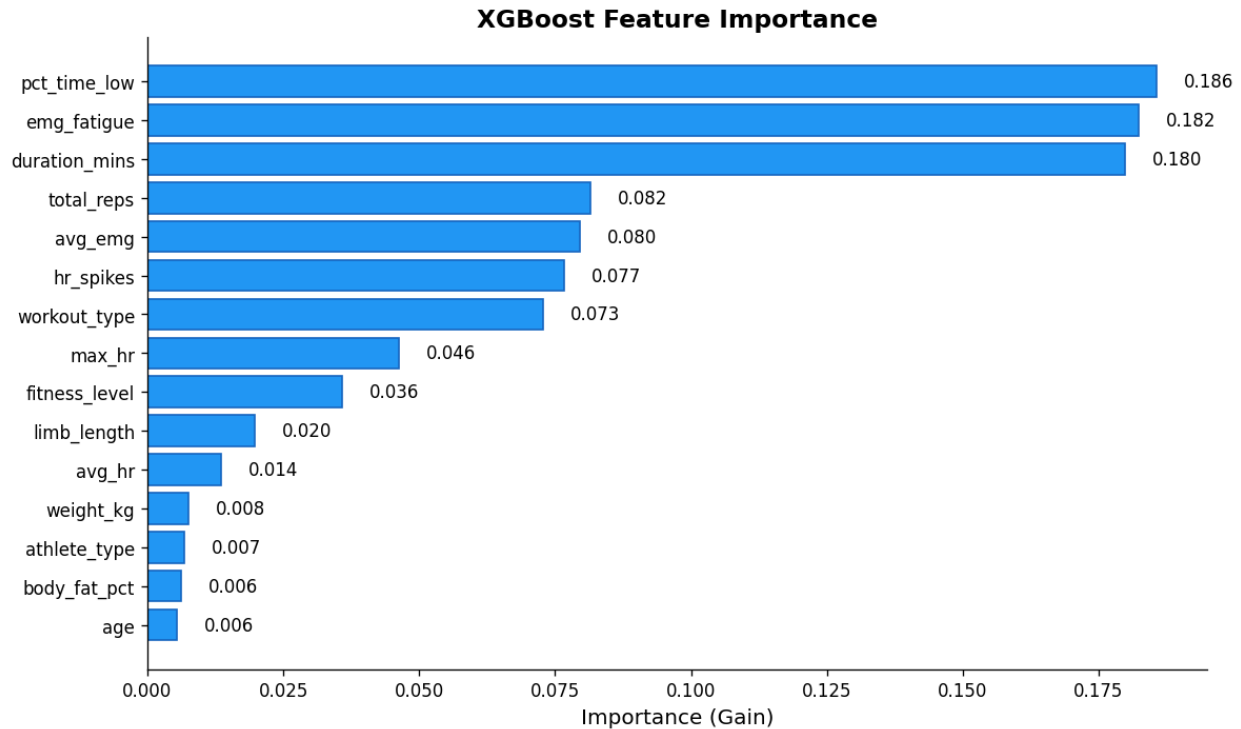
Class	Label	Sub-Profiles	Base Samples	x2.5 Scale	Edge Cases	Total
Low	0	8	4,500	11,250	600	~11,850
Moderate	1	7	4,500	11,250	900	~12,150
High	2	6	4,500	11,250	600	~11,850
Maximum	3	8	4,500	11,250	300	~11,550

Physical constraints are enforced post-sampling: $\text{max_hr} \geq \text{avg_hr}$, $\text{max_hr} \leq 210$, pct_time_low inversely correlated with avg_hr (except for powerlifting profiles where low HR + high EMG is valid).

Training Process

1. Load `synthetic_workouts.csv`
2. Convert 4 categorical columns to pandas category dtype (XGBoost native handling)
3. Stratified 80/20 train/test split (`random_state=42`)
4. RandomizedSearchCV: 50 hyperparameter combinations, 5-fold stratified CV, scored on F1-macro
5. Train final model with best parameters on full training set
6. Evaluate on held-out test set
7. Generate SHAP summary plots (per-class bee swarm + global bar plot)
8. Export: `workout_model.joblib`, `workout_model.json`, `model_config.json`





Inference in Production

When the user triggers analysis from the frontend, the backend's `predictor.py` loads the saved model (singleton, loaded once at startup), constructs a 15-feature `DataFrame`, runs `model.predict()` and `model.predict_proba()`, computes SHAP values via `TreeExplainer`, and returns the prediction with natural-language explanation.

3.8 AI Coach (Gemini Integration)

The AI Coach is a context-aware fitness chatbot powered by Google Gemini 2.5 Flash. When a user opens the chat for an analyzed workout, the backend:

1. Fetches the user's full profile (demographics)
2. Fetches the complete workout data (sensor metrics + AI prediction + SHAP factors)
3. Builds a rich system prompt injecting all context
4. Calls the Gemini REST API with conversation history
5. Returns the model's response

The system prompt instructs Gemini to: reference actual sensor data, tailor advice to the training style, consider the user's fitness level and body composition, and suggest specific improvements for low-effectiveness workouts.

3.9 Metrics: Power Consumption, Range, and Core-App Protocols

Core-App Communication Protocol Summary

Link	Protocol	Direction	Frequency	Latency
PI → AWS	HTTPS/REST	Push (on workout end)	1 per workout	~1-3s
Lambda → Backend	HTTPS/REST	Relay	1 per workout	~2-5s (cold: ~30s)
Frontend → Backend	HTTPS/REST	On-demand	Per user action	~200ms-2s
Backend → Gemini	HTTPS/REST	On-demand	Per chat message	~3-10s

3.10 Difficulties Encountered

Signal Noise and I2C Bus Stability: Because of the movement of the ECG and EMG wires combined with the low-cost hardware configuration, the system initially suffered from severe signal spikes that resulted in erratic readings and I2C bus contention errors. We resolved these hardware communication and noise problems by physically securing the cables tightly against the user's body to eliminate movement-induced interference.

Automated Threshold Calibration: A significant challenge with the ECG and EMG sensors was physiological variance. Every individual has a different resting heart rate and baseline muscle electrical activity. Hardcoding thresholds was unviable, so we engineered automated, dynamic threshold detection algorithms that continuously calibrate to the specific individual using the device, ensuring accurate readings across different users without manual intervention.

MPU6050 Repetition Counting Accuracy: When initially testing the accelerometer rep counting, the algorithm failed to detect all repetitions. We compensated by significantly increasing the sensor's sensitivity, which subsequently caused the opposite problem: severe false positives (phantom reps) from minor body shifts. The solution was implementing a mandatory algorithmic cooldown period of approximately 1.0 seconds (the minimum physiological time required for a controlled repetition), which successfully filtered out the noise while capturing legitimate reps.

SHAP Compatibility with XGBoost 2.0+: The SHAP library's TreeExplainer failed with a ValueError when processing the XGBoost 2.0+ multi-class model format, because the base_score parameter was serialized as an array string (e.g., "[0.25, 0.25, 0.25, 0.25]") instead of a scalar. This was resolved with a monkey-patch that intercepts the UBJSON decoder and extracts the first element of the array before SHAP processes it.

Render Free Tier Cold Starts: The Render free tier spins down the web service after 15 minutes of inactivity. The first request after spin-down takes approximately 30 seconds. The workaround is to access the dashboard URL before running a workout to warm up the service.

4. Testing

4.1 Sensor Testing and Calibration

Each sensor was tested individually before integration into the concurrent multi-threaded loop. Individual python scripts, integrated into the main script, later.

ECG (AD8232): The beat detection algorithm was validated by comparing detected BPM against manual pulse counting over 60-second intervals. The dynamic baseline with 0.99 smoothing factor was tuned to track breathing-induced voltage drift without following R-wave peaks.

EMG: The 2-second calibration phase was validated by comparing noise floor measurements at rest versus during flexion. The noise barrier calculation $((\text{max}-\text{min})/2 + 0.015V)$, capped at 0.145V) was tuned to eliminate resting noise while preserving active muscle signals.

MPU6050: The rep counting state machine was tested with controlled repetitions (bicep curls, squats). The 0.2g deviation threshold and 1.0s cooldown were tuned to minimize both false positives (phantom reps from fidgeting) and false negatives (missed reps during slow movements).

4.2 End-to-End Pipeline Testing

The full data pipeline was tested from sensor reading to dashboard display:

1. Pi reads sensors and computes 9 aggregate features on STOP
2. Pi POSTs JSON to AWS API Gateway (<https://z4mogo62z1.execute-api.eu-north-1.amazonaws.com/prod/workout>)
3. Lambda validates payload and forwards to FastAPI on Render
4. Backend saves workout to Supabase PostgreSQL with status='pending'
5. Dashboard displays new workout card
6. User selects workout type and triggers AI analysis
7. XGBoost predicts effectiveness, SHAP generates explanation
8. AI Coach provides personalized advice based on the analysis

Manual pipeline verification (without Raspberry Pi):

The full AWS → Lambda → Backend pipeline was verified using a direct PowerShell command:

```
Invoke-RestMethod -Uri "https://z4mogo62z1.execute-api.eu-north-1.amazonaws.com/prod/workout" `
  -Method POST -ContentType "application/json" `
  -Body '{ "user_id": "6f228dcf-c876-4276-b454-e2fe2d743223", "duration_mins": 10.0, "avg_hr": 120.0, "max_hr": 155.0, "hr_spikes": 1, "pct_time_low": 25.0, "avg_emg": 300.0, "emg_fatigue": 12.0, "total_reps": 20 }'
```


Response received:

message: Workout data forwarded to backend successfully

workout: @{"id=62bf4d65-0fad-4089-aa64-04b83d29f116; user_id=6f228dcf-c876-4276..."}

This confirms the complete IoT pipeline: PC (simulating Pi) → AWS API Gateway → Lambda → FastAPI Backend → Supabase.

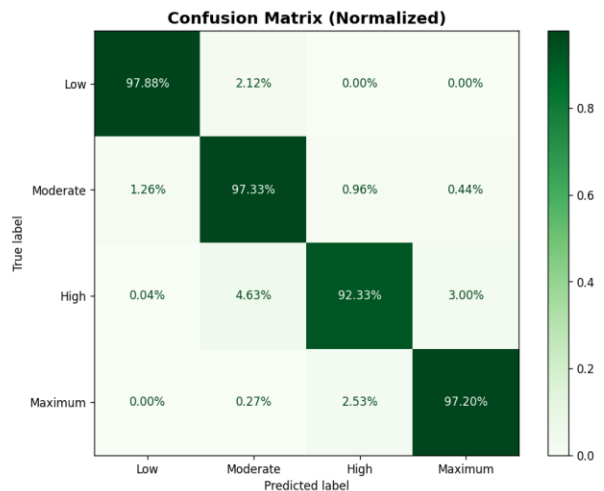
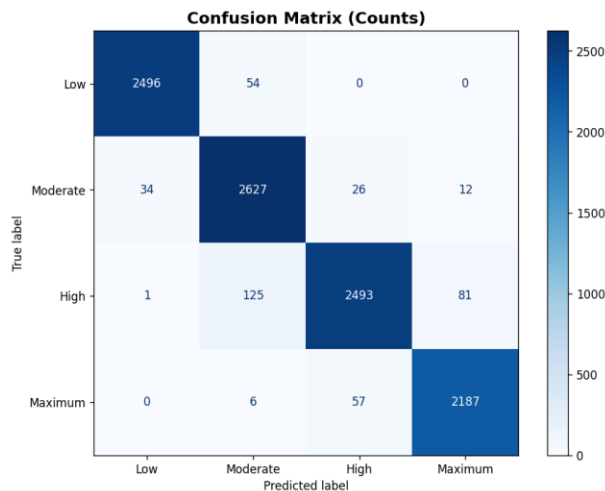
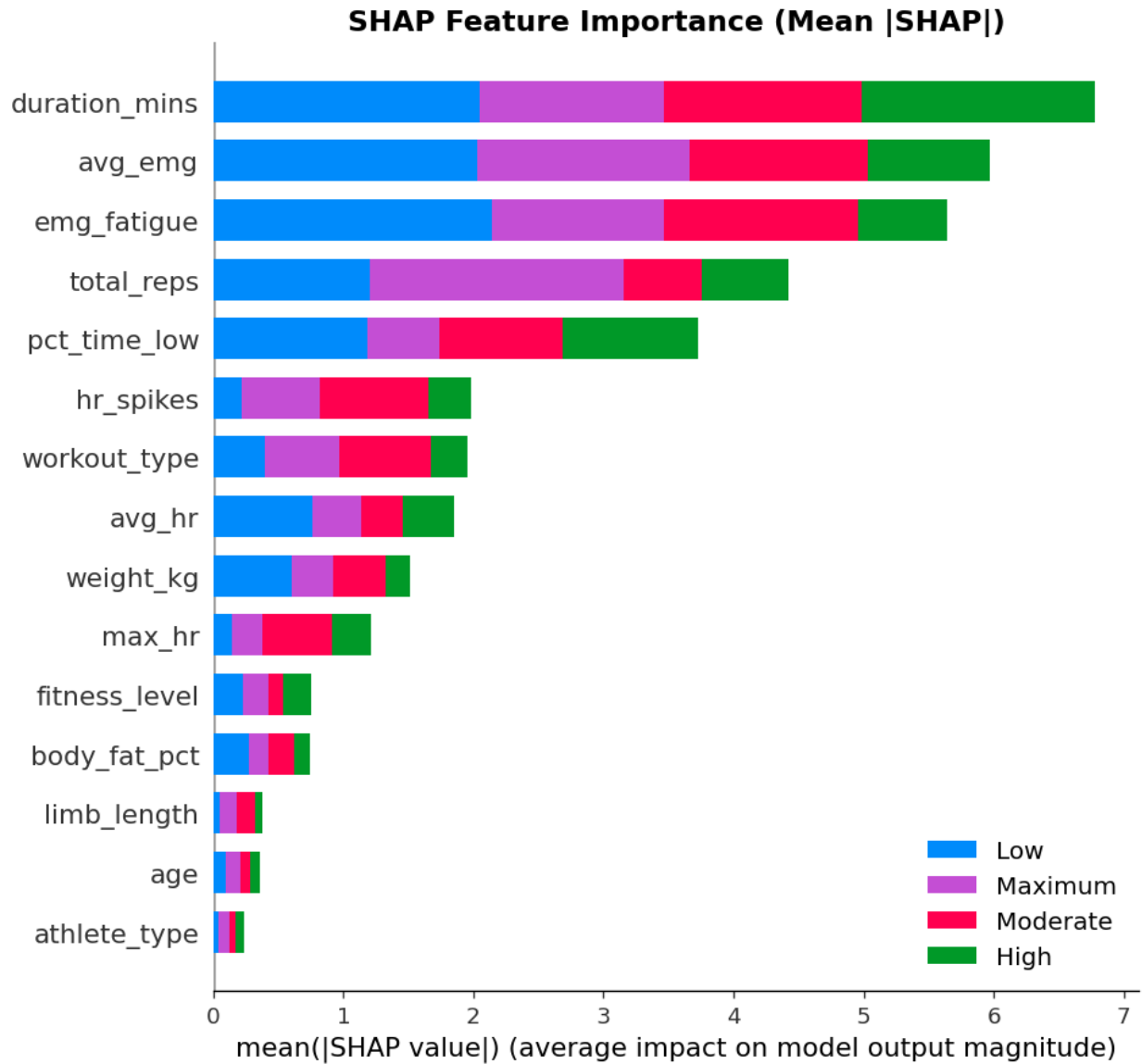
4.3 ML Model Evaluation

The XGBoost model was evaluated on a held-out test set (20% of data, stratified split):

Metric	Value
Test Accuracy	96.12%
Test F1 (macro)	96.16%

Per-class performance:

Class	Precision	Recall	F1-Score	Support
0 (Low)	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]
1 (Moderate)	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]
2 (High)	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]
3 (Maximum)	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]	[USER INPUT REQUIRED]



Demo scenario validation (4 edge cases):

```
Running: Ultra-short burst (3 min, extreme intensity)
[PASS] Expected: 0 (Low ) | Predicted: 0 (Low )
Confidence: 99.9%
Key Factors: short session duration -> 3 min | strong muscle engagement -> 700 (EMG units) | significant muscle fatigue -> 40.0% | low rep count -> 25

Running: Phone scrolling (80 min, zero effort)
[PASS] Expected: 0 (Low ) | Predicted: 0 (Low )
Confidence: 100.0%
Key Factors: lots of low-HR zone time -> 92.0% | minimal muscle fatigue -> 1.5% | weak muscle engagement -> 90 (EMG units) | low avg heart rate -> 82 BPM

Running: Solid bodyweight workout (45 min)
[PASS] Expected: 2 (High ) | Predicted: 2 (High )
Confidence: 100.0%
Key Factors: significant muscle fatigue -> 20.0% | minimal low-HR zone time -> 12.0% | high rep count -> 120 | strong muscle engagement -> 480 (EMG units)

Running: Full beast mode (60 min, everything maxed)
[PASS] Expected: 3 (Maximum ) | Predicted: 3 (Maximum )
Confidence: 95.6%
Key Factors: strong muscle engagement -> 650 (EMG units) | lighter body weight -> 80.0 kg | significant muscle fatigue -> 35.0% | workout type: hypertrophy
```

4.4 Dashboard and API Testing

The REST API endpoints were tested using manual HTTP requests and the React frontend. The dashboard was tested across browsers to verify:

- Workout card rendering with correct status indicators (pending vs analyzed)
- Workout type selection dropdown functionality
- AI analysis trigger and result display (effectiveness gauge, SHAP factors)
- AI Coach chat interaction with conversation history
- Responsive layout on desktop and mobile viewports
- Supabase authentication flow (register, login, JWT token management)

5. Collaboration and AI Assistance

5.1 Version Control

GitHub was utilized throughout the project lifecycle for version control, collaborative development, and secure codebase backup. This enabled seamless integration between the frontend, backend, and hardware scripts developed across different environments.

5.2 LLM Assistance (AI Coding Tools)

Throughout the development process, Large Language Model (LLM) based AI coding assistants were used to accelerate development. These tools were utilized for:

- **Hardware Implementation Guidance:** AI provided initial architectural ideas and troubleshooting strategies during the hardware assembly phase.
- **Code scaffolding:** Generating boilerplate for FastAPI routers, React components, and Pydantic schemas.

- **Debugging:** Diagnosing software issues such as SHAP/XGBoost compatibility and Supabase integration problems.
- **Algorithm refinement:** Tuning the ECG beat detection smoothing factors, EMG noise barrier calibration, and MPU6050 rep counting thresholds.
- **Synthetic data design:** Iterating on sub-profile definitions and edge-case interpolation strategies.
- **Documentation:** Drafting technical documentation and formatting.

Human Intervention and Expert Guidance: While AI tools served as a powerful accelerator, they could not solve the complex physical constraints of our specific Proof of Concept. The unique hardware conflicts (e.g., analog noise from movement vs. accelerometer requirements) required manual engineering and out-of-the-box thinking. Ultimately, we navigated these critical hardware constraints and finalized our implementation strategies through independent problem-solving and crucial hints, ideas, and guidance from our laboratory professor.

All AI-generated code was reviewed, understood, and adapted before integration. The AI served as an assistant, not a replacement for fundamental engineering understanding.

5.3 Machine Learning (XGBoost Classifier)

The core AI component of the project is the XGBoost multi-class classifier for workout effectiveness prediction. This is not a pre-trained or off-the-shelf model — it was custom-designed and trained specifically for this project’s fitness sensor data.

Key ML decisions and their rationale:

Why XGBoost? Workout effectiveness depends on complex conditional rules (e.g., “IF high intensity AND short duration THEN Low”, “IF low HR AND extreme EMG THEN could be Maximum powerlifting”). Decision tree ensembles naturally model these conditional relationships, outperforming neural networks on this type of tabular data with discrete decision boundaries.

Why train on synthetic data? Collecting 51,000 real labeled workouts would require hundreds of users over months. Synthetic data was generated from physiologically grounded sub-profiles (designed from 15 manually labeled real workout scenarios) with physical constraint enforcement to ensure realism. Also, at the moment we do not have the infrastructure to collect and store real-time physiological data from multiple users.

Why 15 features (not just 8 sensor values)? Raw sensor values alone cannot distinguish between a powerlifter (low HR, extreme EMG, few reps = Maximum) and a lazy gym-goer (low HR, low EMG, few reps = Low). The 7 demographic features provide the biological context that makes correct classification possible.

Why SHAP for explainability? Unlike black-box neural networks, TreeSHAP provides exact, polynomial-time Shapley value computation for tree ensemble models. This enables per-

prediction explanations that tell the user specifically which factors drove their effectiveness rating — critical for actionable fitness feedback.

6. Conclusion and Future Work

6.1 Summary

This project successfully implements an end-to-end IoT fitness monitoring system that combines real-time biometric sensor data acquisition from three hardware sensors (ECG, EMG, accelerometer), secure cloud-relayed data transmission via AWS API Gateway and Lambda, persistent storage in a managed PostgreSQL database, machine learning-based workout effectiveness prediction using XGBoost, SHAP-based prediction explainability, and an AI-powered fitness coaching chatbot via Google Gemini.

The XGBoost classifier achieves a test accuracy of 96.12% and F1-macro of 96.16% on the synthetic evaluation set, confirming its ability to distinguish four levels of workout effectiveness across diverse training styles (powerlifting, hypertrophy, endurance, high-intensity). The SHAP integration ensures that every prediction is accompanied by a human-readable explanation identifying the top contributing factors, making the AI analysis actionable rather than opaque.

The system demonstrates practical integration of multiple engineering disciplines: analog electronics (voltage divider design, mixed-voltage safety), digital communication protocols (I2C bus sharing, thread-safe concurrent access), cloud architecture (AWS serverless pipeline), full-stack web development (FastAPI + React + Supabase), and applied machine learning (supervised classification, feature engineering, explainability).

6.2 Limitations

1. **Proof of Concept Form Factor:** The system is a prototype designed to prove the feasibility of raw sensor integration. The Raspberry Pi currently requires a continuous tethered AC power connection, making it impossible to move freely around a commercial gym.
2. **Conflicting Sensor Requirements:** The core physical limitation is the conflict between the sensors. The ECG and EMG modules require near-zero cable movement to avoid extreme signal noise, whereas the MPU6050 accelerometer relies on rigorous physical movement. Even with the hardware secured in a custom box and cables tightly managed, the wearer is effectively limited to stationary, low-travel exercises such as push-ups.
3. **Raw Hardware Bulk:** We deliberately chose to implement raw hardware sensors instead of relying on a pre-made commercial belt to demonstrate analog signal processing and integration. However, this significantly increases bulk, especially the dual 9V battery array and custom voltage divider required for the EMG module.

4. I2C Bus Contention: The I2C bus is shared between two slave devices (the ADS1115 ADC and the MPU6050). Pushing high-frequency concurrent sampling through a single data line requires strict software locking and occasionally triggers bus contention errors.
5. Render Free Tier Cold Starts: The backend spins down after 15 minutes of inactivity. The first HTTP request after spin-down takes approximately 30 seconds to wake the service, occasionally causing the AWS Lambda relay to time out.
6. Single-Device Architecture: The system currently supports one Raspberry Pi session at a time, with no device registry for multi-user hardware scaling.
7. MPU sensor detects reps based on thresholds, that differ from an individual to other. In order to work properly, we need to set a general movement range, and fixed thresholds.

6.3 Future Work

Scalability

- Support multiple Raspberry Pi devices with unique device IDs and per-device calibration profiles
- Implement AWS IoT Device Shadow for device state management and OTA configuration updates
- Implement a device registry for managing multiple monitoring stations
- Migrate from Render free tier to AWS ECS/Fargate for reliable, always-on backend hosting
- Horizontal scaling of the backend with load balancing for concurrent multi-user workouts

Reliability

- Implement local data buffering on the Pi for offline operation (store-and-forward when internet is unavailable)
- Add watchdog timer to automatically restart the sensor loop on crashes
- Implement health monitoring for the Pi (CPU temperature, memory, disk) with cloud reporting
- Implement automated model retraining pipeline triggered by data drift detection
- Add end-to-end encryption for sensor data in transit and at rest

Additional Enhancements

- Design and manufacture a custom, integrated wearable belt to solve the current bulk and movement limitations. This would require a completely different hardware approach (custom PCB, miniaturized sensors) but would allow the system to be worn easily during any type of training.
- Integrate AWS IoT Core with MQTT protocol for real-time streaming of live biometric data to the dashboard

- Add Bluetooth Low Energy (BLE) sensor modules to eliminate wired connections
 - Implement online/adaptive learning to allow the ML model to adjust to individual users without full retraining
 - Add mobile application with push notifications via Firebase Cloud Messaging
 - Implement historical data analysis and trend reporting (weekly/monthly progress charts)
 - Add GPS module for outdoor workout tracking
 - Integrate with third-party fitness APIs (Strava, Apple Health, Google Fit) for data export
-

Github repository: [fitness-monitoring-system](#)

Website: [FitMonitor — AI Workout Analysis](#)

Bibliography:

<https://scikit-learn.org/stable/index.html>

[IoT in Fitness: Internet of Things Benefits, Use Cases, Devices in Gym](#)

[3 Different Body Types and What They Mean for Your Health](#)